

Design Notes

Table of Contents

技术设计思路

结论

这个方案的核心不是“把车写动”，而是先把电控系统拆成可维护的控制链路：输入层负责产生命令，任务调度层负责节奏，底盘和云台负责闭环控制，BSP 层负责对接真实硬件。

总体架构

按键/串口/遥控器

|
v

remote.c 生成 remote_cmd_t

|
v

app.c 定时调度 ControlTask

|

+--> gimbal.c: 目标角 -> PID -> 云台电机输出

|

+--> chassis.c: vx/vy/wz -> 轮速 -> PID -> 底盘电机输出

|
v

bsp_port.c 连接 STM32 HAL、GPIO、TIM、CAN、编码器、IMU

为什么不能把逻辑堆在 main.c

RoboMaster 电控项目后期一定会多人协作：有人调底盘，有人调云台，有人接遥控器，有人调通信。如果所有逻辑都写在 main.c，任何人改一个功能都可能影响其他模块。模块化以后，每个文件只负责自己的对象，调试范围更小，后续接真实硬件也更稳。

状态机设计

整车状态：

- ROBOT_STOP：所有电机输出清零，PID 复位。
- ROBOT_NORMAL：底盘和云台正常工作。

底盘状态：

- CHASSIS_INDEPENDENT：底盘按自身坐标系运动。
- CHASSIS_SPIN：小陀螺，底盘加入固定旋转角速度。
- CHASSIS_FOLLOW_GIMBAL：用 yaw 偏差产生底盘旋转角速度，让底盘朝向追云台。

控制链路

底盘：

$v_x/v_y/w_z$ -> X 型全向轮解算 -> 4 个目标轮速 -> 4 路 PID -> 电机输出

云台：

key2/key3 -> 目标角变化 -> IMU 角度反馈 -> PID -> yaw/pitch 输出

安全保护

优先做三个保护：

- 输出限幅：所有电机命令经过限幅，避免疯转。
- Pitch 限位：目标角限制在 $[-20, 30]$ ，避免打机械限位。
- 急停：按键触发 ROBOT_STOP，所有输出立即清零。

和题目要求的对应关系

- 文件结构：chassis/gimbal/remote/pid/main
- 底盘：X 型全向轮运动学、编码器速度闭环接口
- 云台：yaw/pitch 目标角、pitch 限位、IMU 反馈接口
- 安全：输出限幅、pitch 限位、急停

- 进阶：任务调度、模式切换、串口协议解析框架

Source: C:/Temp/codex_rm_assignment/rm_control_assignment_extracted.md:32